R.V.S SCHOOL OF ENGINEERING AND TECHNOLOGY

DINDIGUL

PROJECT TITLE: WhatNext Vision Motors: Shaping the Future of Mobility with Innovation and Excellence

TEAM LEADER : T.KISHORE KUMAR

kishorebsm291@gmail.com

TEAM MEMBERS:

S.MUKESHKANNA-
kannamukesh428@gmail.com

R.VISHAKAN- vishakanraja2006@gmail.com

S.DINESH -
dineshsureka60@gmail.com

S.HARISH KUMAR –senthilkumarsenthilkumar97497@gmail.com

WhatsNext Vision Motors – Salesforce Project

WhatsNext Vision Motors, a pioneering force in the automotive industry, is dedicated to transforming the mobility sector with innovative technology and solutions that prioritize customer needs. The company has embarked on an ambitious Salesforce project with the core objective of enhancing the customer experience and streamlining its operational processes.

At the heart of this project is the improvement of the customer ordering process. The system is designed to automatically suggest the nearest dealer location to customers based on their address. This feature is intended to significantly enhance the convenience and efficiency of the ordering experience, making it more customer-friendly and reducing the time and effort required from the customer's end.

The project also addresses a common issue in the automotive industry: stock availability. The system includes a mechanism that prevents customers from placing orders for vehicles that are out of stock. This proactive approach ensures that customers can only create orders for vehicles that are currently available, thus avoiding potential confusion and disappointment that may arise from stock unavailability. This feature not only enhances customer satisfaction but also improves the accuracy of the company's order fulfillment process.

Furthermore, the project incorporates a scheduled process for updating the status of bulk order records. This automated process is designed to update the order status based on stock availability. If a vehicle is out of stock at the time of order placement, the system will update the order status to 'Pending.' On the other hand, if the vehicle is in stock, the system will update the status to 'Confirmed.' This ensures that all orders are accurately reflected in terms of their fulfillment status, providing clear and transparent communication to customers regarding the status of their orders.

The implementation of this Salesforce project at WhatsNext Vision Motors is expected to yield several benefits. It aims to create a more efficient ordering system that reduces the potential for errors and improves the overall service provided to customers. By streamlining the ordering process and ensuring accurate stock availability, the company can enhance customer satisfaction and loyalty.

Moreover, the project is expected to contribute to operational efficiency by reducing the administrative burden on staff. By automating certain processes, employees can focus on more strategic tasks that require human intervention and expertise. This not only improves the overall productivity of the company but also allows for a more agile response to market demands and customer needs.

Creating Developer Account & Account Activation

Creating Developer Account

Creating a developerorginSalesforce.

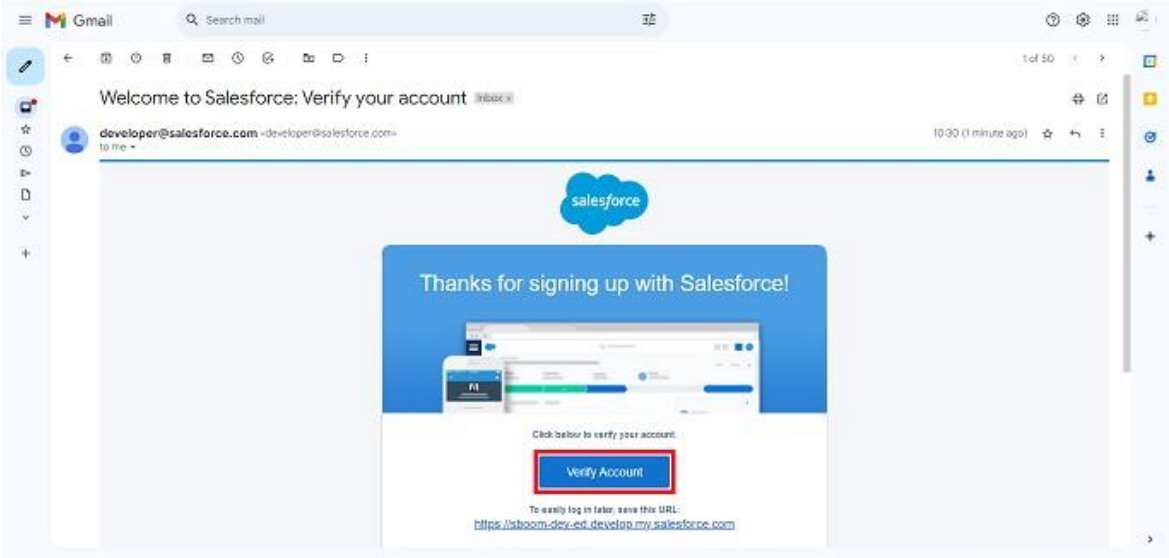
Go to https://developer.salesforce.com/signup

On the sign up form, enter the following details:

- First name & Last name
- Email
- Role: Developer
- Company: College Name
- Country: India
- Postal Code: PIN code
- Username: should be a combination of your name and company

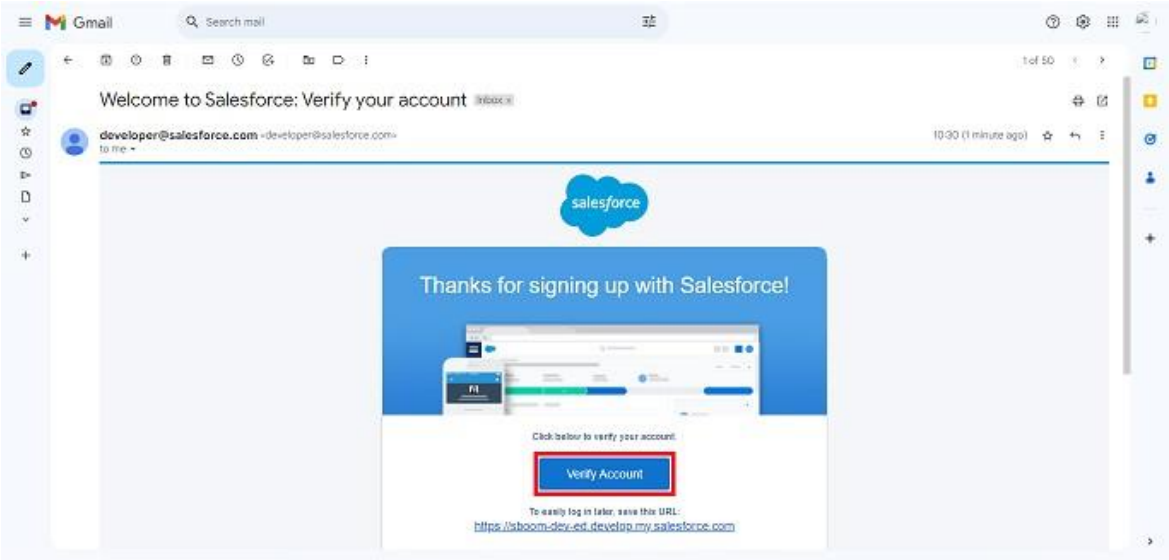
This need not be an actual email ID; you can give anything in the format: username@organization.com Click

on Sign Me Up after filling these details.

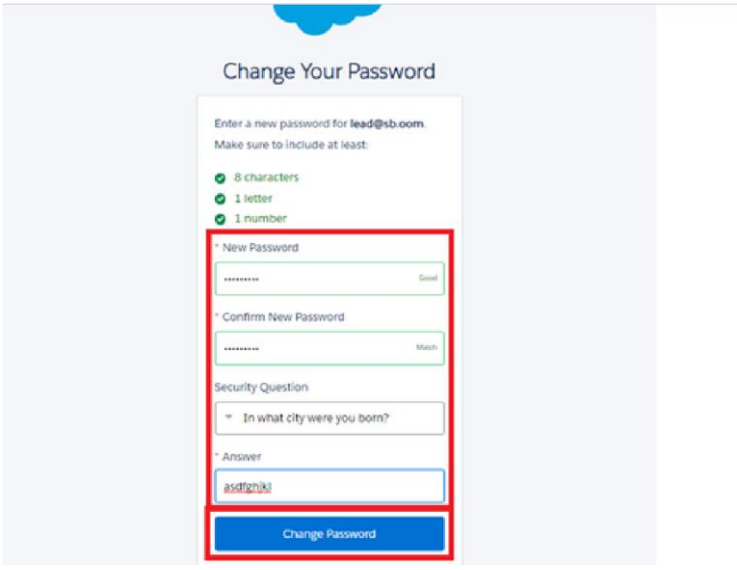


Account Activation

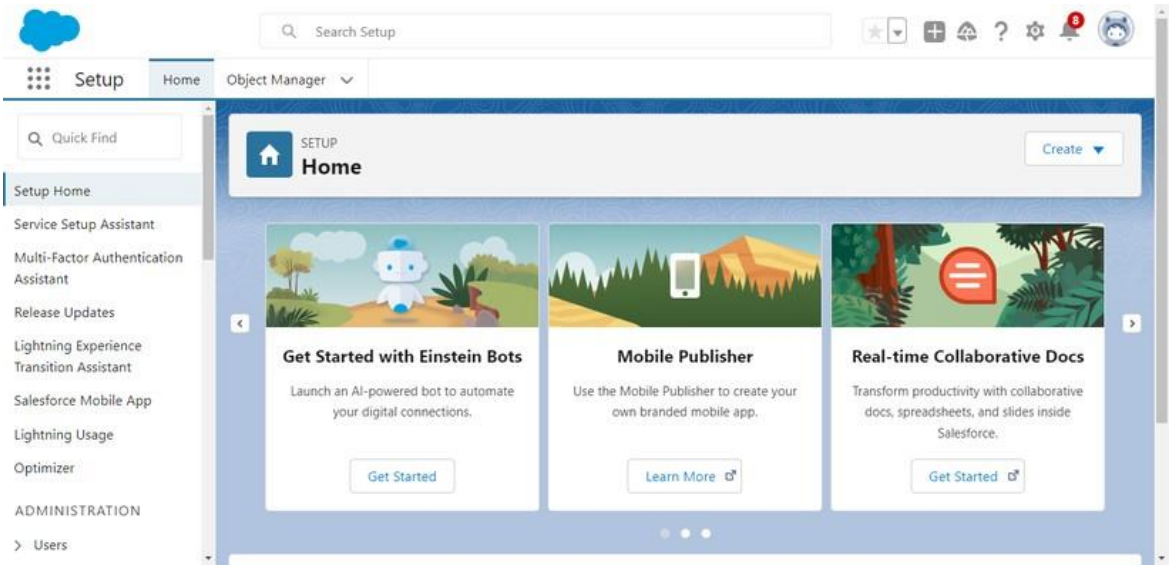
Go to the inbox oftheemailthatyou used while signing up. Click on Verify Account to activate your account. The email may take 5–10 minutes.



Click on Verify Account.



Give a password and answer a security question and click on Change Password.



Then you will redirect to your Salesforce Setup page.

Data Management – Objects

Objects & Relationships

Object Name	Purpose	Relationships
Vehicle__c	Stores vehicle details	Related to Dealer & Orders
Vehicle_Dealer__c	Stores authorized dealer info	Related to Orders
Vehicle_Customer__c	Stores customer details	Related to Orders & Test Drives
Vehicle_Order__c	Tracks vehicle purchases	Related to Customer & Vehicle
Vehicle_Test_Drive__c	Tracks test drive bookings	Related to Customer & Vehicle
Vehicle_Service_Request__c	Tracks vehicle servicing requests	Related to Customer & Vehicle

Create Vehicle Object

The purpose of creating a Vehicle custom object is to have a clear picture of the vehicle details.

To create an object:



- From the setup page → Click on Object Manager → Click on Create → Click on Custom Object.
- Enter the label name → Vehicle.
- Plural label name → Vehicles.
- Enter Record Name Label and Format.
- Record Name → Vehicle Name.
- Data Type → Text.
- Click on Allow reports.
- Allow search → Save.

Create Dealer Object

The purpose of creating a Dealer object is to have detailed information about Dealer.

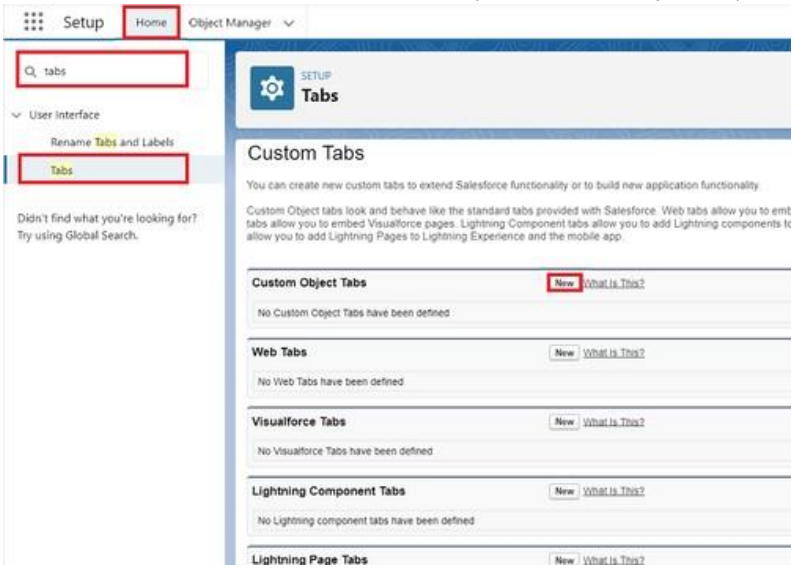
To create an object:

- From the setup page Click on Object Manager Click on Create Click on Custom Object.
- Enter the label name Vehicle Dealer.
- Plural label name Vehicle Dealers.
- Enter Record Name Label and Format.
- Record Name Dealer Name.
- Data Type Text.
- Click on Allow reports.
- Allow search Save.

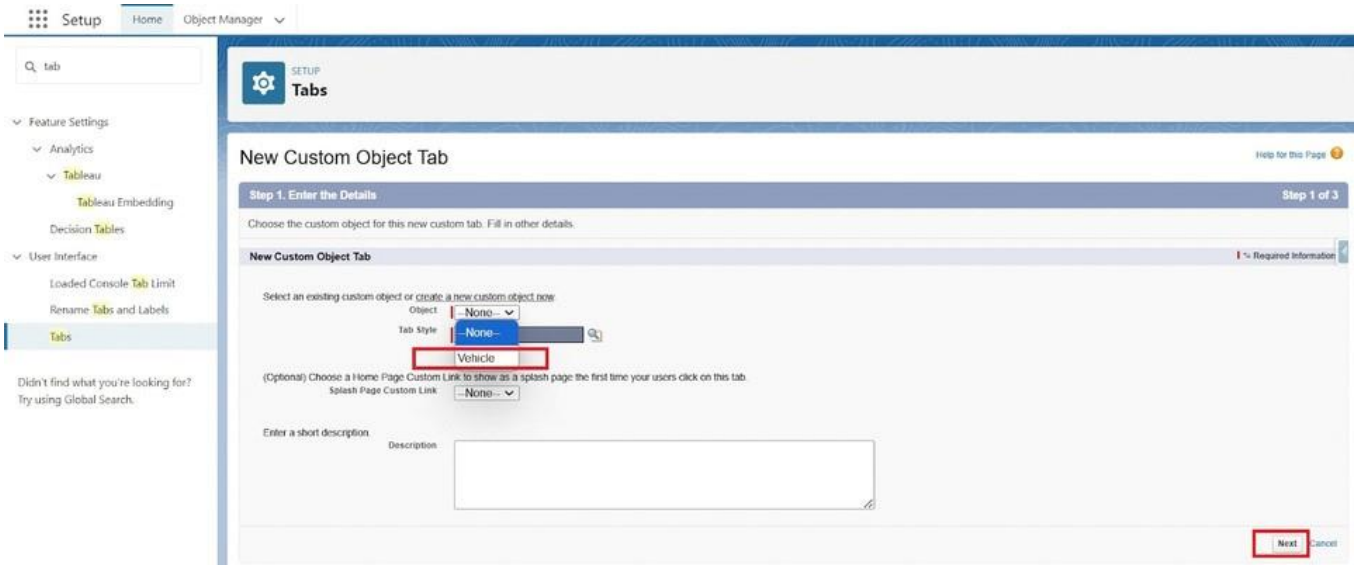
Creating a Custom Tab (Vehicle)

Page 6

Go to setup page → type Tabs in Quick Find bar → click on Tabs → New (under custom object tab).



Select Object (Vehicle) → Select any tab style → Next (Add to profiles page) keep it as default → Next (Add to Custom App) keep it as default → Save.

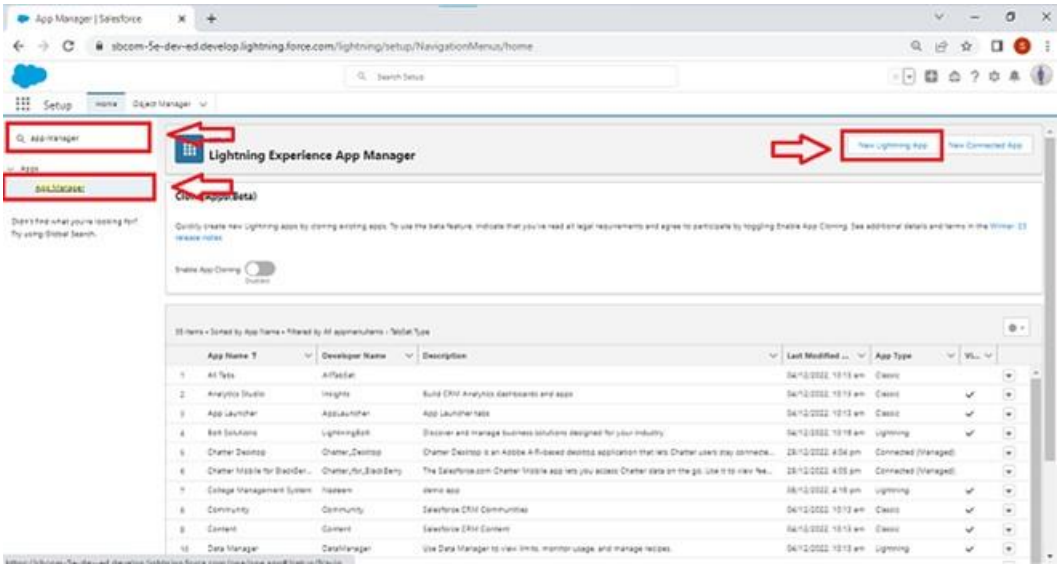


Create a Lightning App

Page 7

To create a lightning app page:

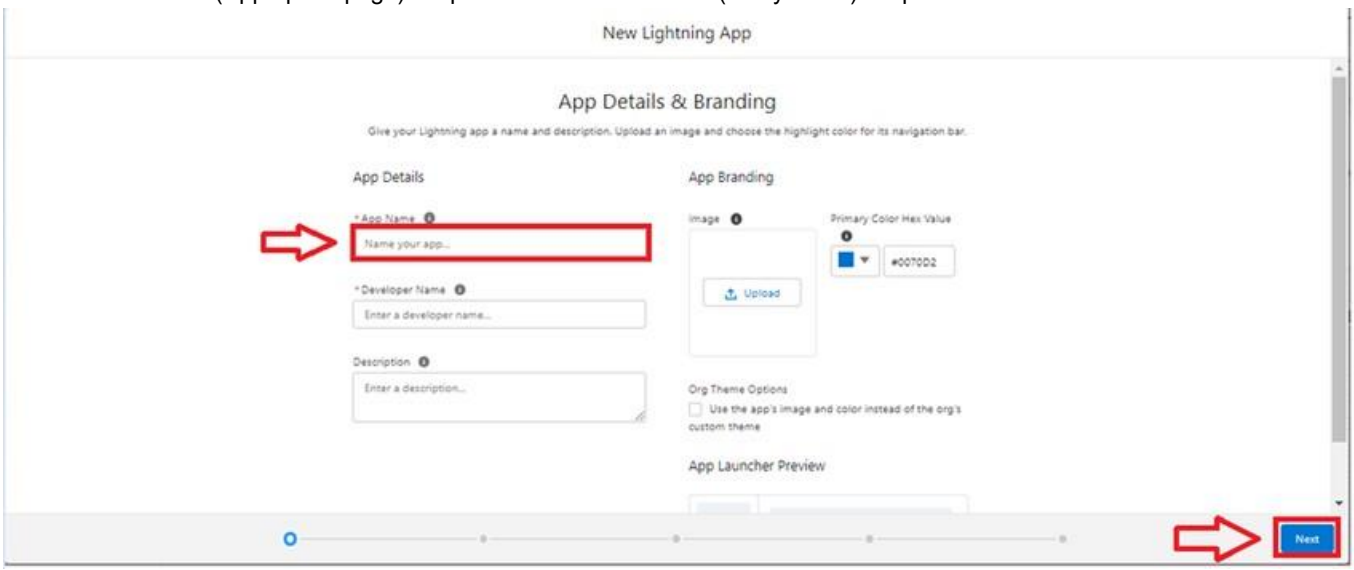
Go to setup page → search “app manager” in quick find → select “app manager” → click on New lightning App.



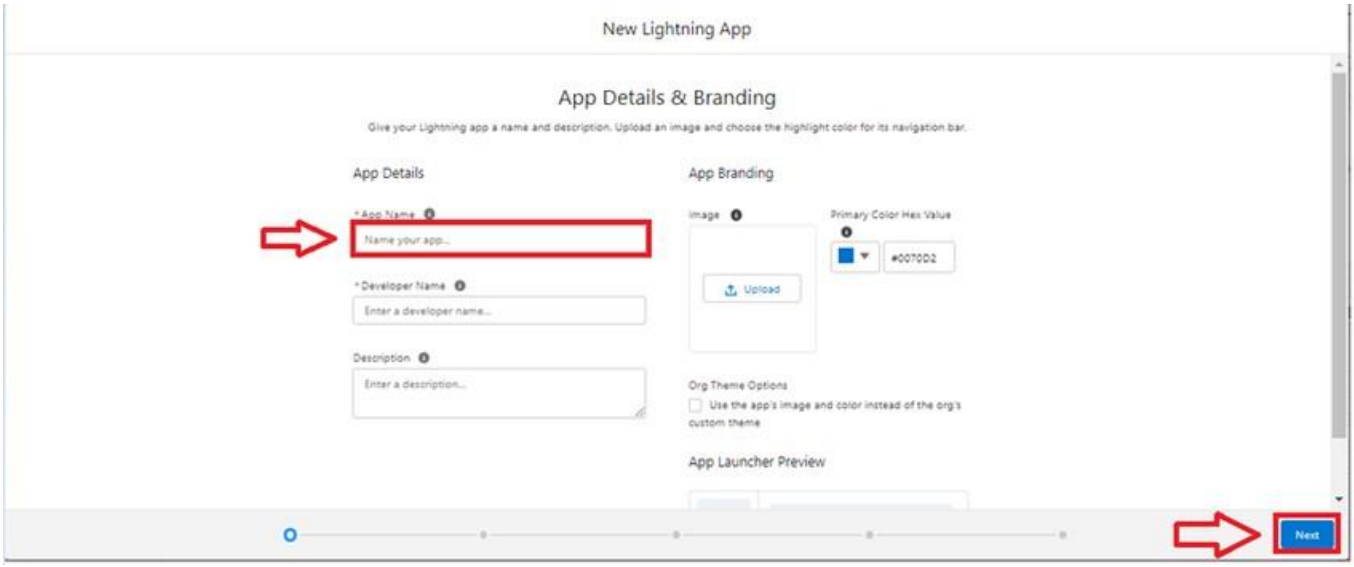
Fill the app name in app details and branding as follow

- App Name: WhatNext Vision Motors
- Developer Name: this will auto populated
- Description: Give a meaningful description
- Image: optional (if you want to give any image you can otherwise not mandatory)
- Primary color hex value: keep this default

Then click Next → (App option page) keep it as default → Next → (Utility Items) keep it as default Next.



To Add Navigation Items:

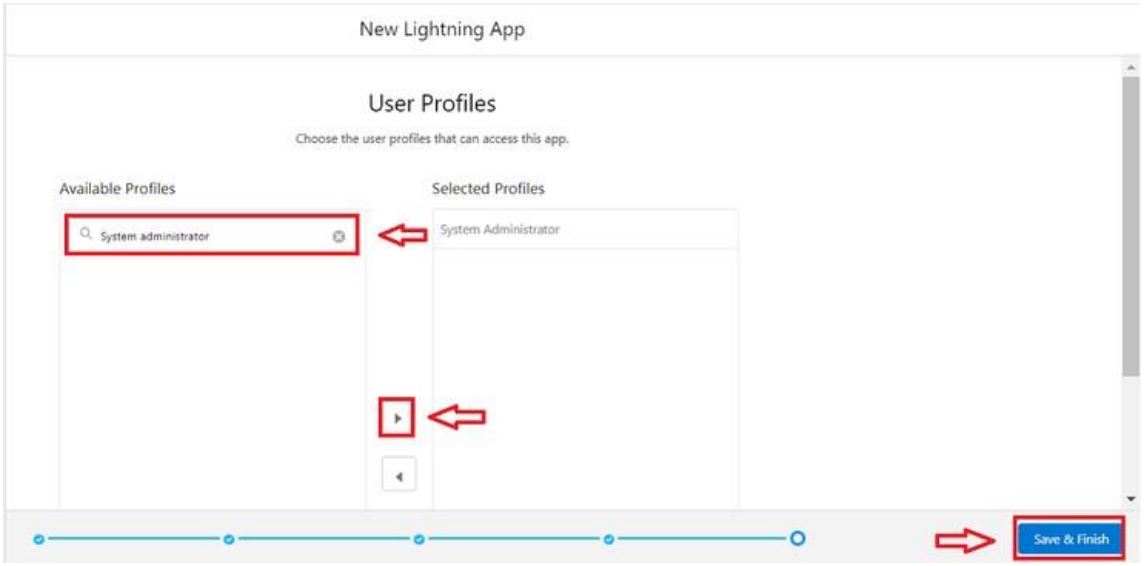


Search the items in the search bar (Vehicle, Dealer, Customer, Order, Test Drive, Service Request, Reports, Dashboard) and move it using the arrow button → Next.

Note: select the custom object which we have created in the previous activity.

To Add User Profiles:

Search profiles (System administrator) in the search bar click on the arrow button save & finish.



Fields & Relationships

Key Fields for Each Object

1. Vehicle__c (Custom Object)

- Vehicle_Name__c (Text)
- Vehicle_Model__c (Picklist: Sedan, SUV, EV, etc.)
- Stock_Quantity__c (Number)
- Price__c (Currency)
- Dealer__c (Lookup to Dealer__c)
- Status__c (Picklist: Available,OutofStock,Discontinued)

2. Vehicle_Dealer__c (Custom Object)

- Dealer_Name__c(Text)
- Dealer_Location__c (Text)
- Dealer_Code__c (Auto Number)
- Phone__c (Phone)
- Email__c (Email)

3. Vehicle_Order__c (Custom Object)

- Customer__c(Lookup toCustomer__c)
- Vehicle__c (Lookup to Vehicle__c)
- Order_Date__c (Date)
- Status__c(Picklist:Pending,Confirmed,Delivered,Canceled)

4. Vehicle_Customer__c (Custom Object)

- Customer_Name__c(Text)
- Email__c (Email)
- Phone__c (Phone)
- Address__c (Text)
- Preferred_Vehicle_Type__c(Picklist:Sedan,SUV,EV,etc.)

5. Vehicle_Test_Drive__c (Custom Object)

- Customer__c(Lookup toCustomer__c)
- Vehicle__c (Lookup to Vehicle__c)
- Test_Drive_Date__c (Date)
- Status__c(Picklist: Scheduled,Completed,Canceled)

6. Vehicle_Service_Request__c (Custom Object)

- Customer__c(Lookup toCustomer__c)
- Vehicle__c (Lookup to Vehicle__c)
- Service_Date__c (Date)
- Issue_Description__c (Text)
- Status__c (Picklist: Requested, In Progress, Completed)

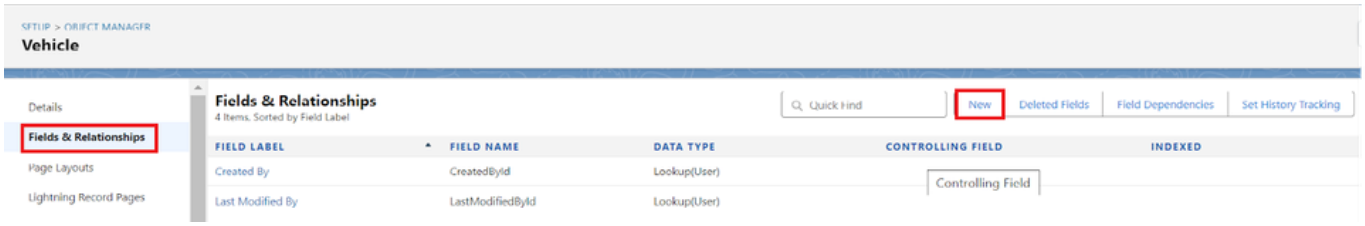
Creating a Field in Vehicle Object

Step 1

Gotosetup click on Object Manager type object name (Vehicle) in quick find bar click on the object.

Step 2

Nowclickon “Fields & Relationships” New.



Step 3

SelectData type as “PickList”.

Step 4

Click on Next.

Step 5

Filltheabove as following: Field Label: Vehicle Model. Fill PickList Values.

Step 6

ClickonNext Next Save and new.

Creating Stock Quantity field in Vehicle Object

Step 1

Repeatthe steps 1 & 2 from the activity 1.

Step 2

SelectData type as “Number”.

Step 3

Click on Next.

Step 4

Filltheabove as following: Field Label: Stock Quality.

Step 5

ClickonNext Next Save and new.

Creating a Lookup Relationship in Vehicle Object to Dealer Object

Step 1

SelectDataType as Lookup Relationship.

Step 2

Click Next.

Step 3

SelectDealer Object Form Related Filed.

Step 4

ClickNext and Save.

Creating a Record-Triggered Flow to Assign Nearest Dealer

Creating a record triggered flow to assign nearest dealer to the customer’s location

Step 1

In Quick Find, type Flows and click on Flows. Click New Flow.

Step 2

Select Start From Scratch and click Next.

Step 3

Select Record-Triggered Flow and click Create

Step 4

Select Vehicle Order Object. Trigger the Flow When: Select Record is Created. Set Entry Condition: All Conditions Are Met (AND); Field: Status__c; Operator: Equals; Value: Pending

---

Step 5

Click + Select Get Records

Step 6

Label: Get Customer Information. Object: Vehicle Customer. Condition: Field: Id; Operator: Equals; Value: {!$Record.Vehicle_Customer__c }

Step 7

Click + → Select Get Records. Label: Get Nearest Dealer. Object: Vehicle Dealer. Condition: Field: Dealer_Location__c; Operator: Equals; Value: {!Get_Customer_Information.Address__c}

Step 8

Click + → Select Update Records. Label: Assign Dealer to Order. How to Find Records to Update and Set Their Values: Use the IDs and all field values from a record or record collection. Select Record(s) to Update: {!Get_Nearest_Dealer}

Step 9

Click Save and Give label Name and Activate Flow. Label Name: Auto Assign Dealer

---

Step 10

Activate Flow

Record-Triggered Flow – Test Drive Reminder

Creating record triggered flow to send an email to the customer reminding about the test drive.

Step 1

Select Record-Triggered Flow and click Create

Step 2

Select Vehicle Test Drive Object. Trigger the Flow When: A record is created or updated. Set Entry Condition: All Conditions Are Met (AND); Field: Status__c; Operator: Equals; Value: Scheduled

---

Step 3

Click + Add Scheduled Paths (below the trigger). Label: Reminder Before Test Drive. Time Source: Test_Drive_Date__c. Offset Number: 1. Offset Options: Days Before. Click Done.

Step 4

Click + Add Element → Get Records. Label: Get Customer Information. Object: Vehicle_Customer__c. Filter Conditions: Id = {!$Record.Customer__c}. How Many Records to Store: Select Only the first record. How to Store Record Data: Choose Automatically store all fields.

Step 5: Send Reminder Email

Click + Add Element → Action. Action Type: Send Email. Label: Send Test Drive Reminder. Subject: “Reminder: Your Test Drive is Tomorrow!”. Recipient Address List: {!Get_Customer_Information.Email__c}. Rich-Text-Formatted Body: True. Body: Create Variable. API Name: EmailSent

---

Step 6

Clicksave. Label Name: Test Drive Reminder

Step 7

Activate Flow

Flow Page After Creation:

Test your Flow:

Create Apex and Trigger Batch Jobs

Step 1

ClickDeveloper Console From Gear icon

Step 2

clickFilethen Select New then Select Apex.

Step 3

GiveName For Apex Class. Class Name: VehicleOrderTriggerhandler

---

Step 4: Write Apex Code

```
public class VehicleOrderTriggerHandler  {

    public static void handleTrigger(List<Vehicle_Order__c> newOrders, Map<Id, Vehicle_Order__c> oldOrders, Boolean isBefore, Boolean isAfter, Boole

        if(isBefore) {
            if(isInsert || isUpdate) {
                preventOrderIfOutOfStock(newOrders);
            }
        }
        if(isAfter) {
            if(isInsert || isUpdate) {
                updateStockOnOrderPlacement(newOrders);
            }
        }
    }

    //Methodto   prevent orders when the vehicle is out of stock
    privatestatic  void preventOrderIfOutOfStock(List<Vehicle_Order__c> orders) {
        Set<Id>vehicleIds = new Set<Id>();
        for(Vehicle_Order__c order : orders) {
        if(order.Vehicle__c != null) {
                vehicleIds.add(order.Vehicle__c);
            }
        }
        if(!vehicleIds.isEmpty()) {
            Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
            for(Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM Vehicle__c WHERE Id IN :vehicleIds]) {
                vehicleStockMap.put(vehicle.Id, vehicle);
            }
            for(Vehicle_Order__c order : orders) {
```

```
                if (vehicleStockMap.containsKey(order.Vehicle__c)) {
                    Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c); if
                    (vehicle.Stock_Quantity__c <= 0) { order.addError('This vehicle is out of
                    stock. Order cannot be placed.'); }
                }
            }
        }
    }

    //Methodto        update vehicle stock when an order is placed privatestatic void
    updateStockOnOrderPlacement(List<Vehicle_Order__c> orders) {
        Set<Id>vehicleIds = new Set<Id>(); for(Vehicle_Order__c order :
        orders) { if(order.Vehicle__c != null && order.Status__c ==
        'Confirmed') { vehicleIds.add(order.Vehicle__c);
            }
        }
        if(!vehicleIds.isEmpty()) {
            Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
            for(Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM Vehicle__c WHERE Id IN :vehicleIds]) {
                vehicleStockMap.put(vehicle.Id, vehicle);
            }
            List<Vehicle__c> vehiclesToUpdate = new List<Vehicle__c>();
            for(Vehicle_Order__c order : orders) {
                if (vehicleStockMap.containsKey(order.Vehicle__c))  {
                    Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
                    if (vehicle.Stock_Quantity__c > 0)  {
                       vehicle.Stock_Quantity__c -= 1;
                       vehiclesToUpdate.add(vehicle);
                    }
                }
            }
            if(!vehiclesToUpdate.isEmpty()) {
                update vehiclesToUpdate;
            }
        }
    }
}
```

Step 5

WriteTrigger Handler

Step 6

WriterTrigger Class Name and Select Vehicle Order Object

Step 7: Call Apex Class in Trigger Class

```
trigger VehicleOrderTrigger on Vehicle_Order__c (before insert,before update, after insert, after update) {

    VehicleOrderTriggerHandler.handleTrigger(Trigger.new, Trigger.oldMap, Trigger.isBefore,Trigger.isAfter, Trigger.isInsert, Trigger.isUpdate);

}
```

Step 8: Create Batch Job

```
SELECT Id, Status_c, Vehicle___c

FROM Vehicle_Order___c

WHERE Status__c

'Pending'
```

A customer places an order, but the vehicle is out of stock. The order remains pending. After new stock is added, the batch job updates the order to confirmed.

```
global class VehicleOrderBatch implements Database.Batchable<sObject> {

global Database.QueryLocator start (Database.BatchableContext bc) {

return Database.getQueryLocator([
]); } global void execute(Database.BatchableContext bc, List<Vehicle_Order_c>

orderList) { Set<Id> vehicleIds = new Set<Id>(); for (Vehicle_Order_c order

orderList) { if (order.Vehicle_c != null) { vehicleIds.add(order.Vehicle__c);

} } if

(!vehicleIds.isEmpty()) {

Map<Id, Vehicle___c> vehicleStockMap = new Map<Id, Vehicle___c>(); for (Vehicle_c vehicle:

[SELECT Id, Stock Quantity_c FROM Vehicle_c WHERE Id IN vehicleIds]) {

vehicleStockMap.put(vehicle. Id, vehicle);

}

List<Vehicle_Order_c> ordersToUpdate = new List<Vehicle_Order_c>(); List<Vehicle__c> vehiclesToUpdate = new List<Vehicle__c>();

for (Vehicle_Order_c order orderList) { if (vehicleStockMap.containsKey(order. Vehicle__c)) { Vehicle_c vehicle =

vehicleStockMap.get(order. Vehicle__c); if (vehicle.Stock_Quantity___c > 0) { order.Status___c = 'Confirmed';

vehicle.Stock_Quantity___c -= 1; ordersToUpdate.add(order); vehiclesToUpdate.add(vehicle);

}

} } if

(!ordersToUpdate.isEmpty()) {

update ordersToUpdate;

} if (!vehicles ToUpdate.isEmpty())

{ update vehiclesToUpdate;

} } global void finish(Database.BatchableContext bc)

{ System.debug('Vehicle order batch job

completed.');

}
```

Step 8 – Vehicle Order Batch Job Screenshot

Code Coverage: None | API Version: 62

```
global class VehicleOrderBatch implements Database.Batchable<sObject> {

    global Database.QueryLocator start(Database.BatchableContext bc) {
        return Database.getQueryLocator([
            SELECT Id, Status__c, Vehicle__c
            FROM Vehicle_Order__c
            WHERE Status__c = 'Pending'
        ]);
    }

    global void execute(Database.BatchableContext bc, List<Vehicle_Order__c> orderList) {
        Set<Id> vehicleIds = new Set<Id>();

        for (Vehicle_Order__c order : orderList) {
            if (order.Vehicle__c != null) {
                vehicleIds.add(order.Vehicle__c);
            }
        }

        if (!vehicleIds.isEmpty()) {
            Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
            for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM Vehicle__c WHERE Id IN :vehicleIds]) {
                vehicleStockMap.put(vehicle.Id, vehicle);
            }

            List<Vehicle_Order__c> ordersToUpdate = new List<Vehicle_Order__c>();
            List<Vehicle__c> vehiclesToUpdate = new List<Vehicle__c>();

            for (Vehicle_Order__c order : orderList) {
                if (vehicleStockMap.containsKey(order.Vehicle__c)) {
```

```
}
```

Step 9: Create Schedule Class then invoke Batch Class in Schedule Class

```
global class VehicleOrderBatchScheduler implements Schedulable {
    global void execute(SchedulableContext sc) {
        VehicleOrderBatch batchJob = new VehicleOrderBatch();
        Database.executeBatch(batchJob, 50); // 50 is the batch size
    }
}
```

Step 10: Schedule the Batch Job

To run the batch job every night at midnight:

```
String cronExp = '0 0 12 * * ?'; // Runs daily at 12:00 PM
System.schedule('Daily Vehicle Order Processing', cronExp, new VehicleOrderBatchScheduler());
```

Step 11: You Can Check where this Schedule Job is Running

Thank you

WhatsNext Vision Motors | Salesforce Project Guide

Step 11 – Scheduled Jobs Screenshot

